

Raytracing via Subgraph Algorithmus: Szenegraph-Traversierung, Strahlschnitt-Isomorphie und optimale Schattenberechnung

Stephan Epp

9. Juni 2026

Inhaltsverzeichnis

1	Einführung	3
1.1	Beiträge dieser Arbeit	3
2	Mathematische Grundlagen	3
2.1	Szenegraph und geometrische Objekte	3
2.2	Der Subgraph Algorithmus	4
3	Strahlschnitt-Berechnung als Subgraph-Problem	5
3.1	Formale Reduktion	5
3.2	Beispielrechnung: Strahlschnitt in 5-Objekt-Szene	6
3.3	Mehrfachstrahlen und Bildgenerierung	7
4	Schattenberechnung via transitivem Subgraph-Abschluss	8
4.1	Formale Definition des Schattenwurfs	8
5	Reflexion und Brechung als Pfad-Subgraphen	8
5.1	Modellierung von Mehrfachreflexionen	8
5.2	Brechung und Snellsches Gesetz	10
6	Komplexitätsanalyse und Vergleich	10
6.1	SETH-Optimalität	10
6.2	Gerenderte Szene	11
7	Zusammenfassung	11
8	Ausblick	12
8.1	Erweiterung auf Pathtracing und Global Illumination	12
8.2	Echtzeit-Raytracing und GPU-Architektur	13
8.3	Dynamische Szenen und inkrementelle Updates	13
8.4	Verbindung zu Augmented und Mixed Reality	13
8.5	Spektrales Raytracing und Wellenlängenabhängigkeit	14

8.6	Neuronale Rendering-Methoden und NeRF	14
8.7	Prozedurale Generierung und L-Systeme	14
8.8	Formale Verifikation von Renderern	14
8.9	Quantenraytracing	15
8.10	Offene Fragen	15

1 Einführung

Raytracing ist das physikalisch motivierte Standardverfahren zur fotorealistischen Bildsynthese in der Computergrafik (Whitted, 1980). Ausgehend von einer Kamera werden Strahlen durch jeden Bildpixel in eine 3D-Szene geschossen; für jeden Strahl werden Schnitte mit Szenenobjekten, Reflexionen, Brechungen und Schattenwurf berechnet. Die algorithmische Kernaufgabe ist die effiziente Traversierung des *Szenegraphen* $G_S = (V, E)$, dessen Knoten Objekte und Kanten räumliche Nachbarschaftsbeziehungen kodieren.

Klassische Beschleunigungsstrukturen wie *Bounding Volume Hierarchies* (BVH) (Rubin und Whitted, 1980) und *kd-Trees* (Bentley, 1975) reduzieren die Traversierungskomplexität auf $O(n \log n)$, bieten jedoch keine formalen Korrektheitsgarantien bezüglich struktureller Szenenisomorphismen und erfordern aufwändige Neuaufbau-Prozeduren bei dynamischen Szenen.

Der *Subgraph Algorithmus* (Epp, 2026) entscheidet durch Signatur-Arrays und zyklische Rotationen, ob ein Graph G als Subgraph in einem Graphen G' enthalten ist, in Zeit $\Theta(n^3)$ — optimal unter der Strong Exponential Time Hypothesis (SETH). Die zentrale These dieser Arbeit lautet:

Die Schnittberechnung zwischen einem Strahl und einer 3D-Szene, die Schattenberechnung via transitivem Abschluss und die Modellierung von Reflexions-/Brechungsketten lassen sich formal korrekt auf das Subgraph-Isomorphieproblem reduzieren und in $O(n^3)$ lösen.

1.1 Beiträge dieser Arbeit

1. **Szenegraph-Modell:** Formale Definition des Szenegraphen und Nachweis der Isomorphieinvarianz unter Starrkörpertransformationen (Lemma 2.1).
2. **Strahlschnitt-Reduktion:** Reduktion des Strahlschnittproblems auf Subgraph-Isomorphie mit Korrektheit- und Vollständigkeitsbeweis (Satz 3.1, Satz 3.2).
3. **Schattenberechnung:** Formale Behandlung des Schattenwurfs über transitiven Subgraph-Abschluss (Satz 4.1).
4. **Reflexions-/Brechungsketten:** Modellierung von Mehrfachreflexionen als Pfad-Subgraphen mit Tiefenschranke (Satz 5.1).
5. **Komplexitätsanalyse:** SETH-Optimalität und Vergleich mit BVH, kd-Tree und Embree (Satz 6.1, Tabelle 1).

Alle Ergebnisse werden formal bewiesen und durch 7 Matplotlib-Abbildungen illustriert.

2 Mathematische Grundlagen

2.1 Szenegraph und geometrische Objekte

Definition 2.1 (3D-Szene). Eine *3D-Szene* ist ein Tupel $\mathcal{S} = (\mathcal{O}, \mathcal{L}, \mathcal{C})$ mit:

- $\mathcal{O} = \{O_1, \dots, O_m\}$: Menge geometrischer Objekte (Kugeln, Dreiecke, Ebenen, Quader),

- $\mathcal{L} = \{L_1, \dots, L_k\}$: Menge von Lichtquellen,
- $\mathcal{C}$: Kamera mit Position, Blickrichtung und Bildebene.

Definition 2.2 (Szenegraph). Der *Szenegraph* der Szene $\mathcal{S}$ ist ein ungerichteter Graph $G_S = (V_S, E_S)$ mit $V_S = \mathcal{O} \cup \mathcal{L} \cup \{\mathcal{C}\}$ und

$$E_S = \{(v_i, v_j) \mid \text{bbox}(O_i) \cap \text{bbox}(O_j) \neq \emptyset \text{ oder } \|c_i - c_j\|_2 \leq r_{\text{knn}}\},$$

wobei $\text{bbox}(O_i)$ die Bounding Box von O_i und c_i den Schwerpunkt von O_i bezeichnet.

Definition 2.3 (Strahl). Ein *Strahl* ist eine parametrisierte Halbgerade $r(t) = o + t d$ mit Ursprung $o \in \mathbb{R}^3$, normierter Richtung $d \in S^2$, $\|d\|_2 = 1$, und $t \in [t_{\min}, t_{\max}] \subset \mathbb{R}_{\geq 0}$.

Definition 2.4 (Strahl-Subgraph). Sei $r(t)$ ein Strahl. Die Menge der vom Strahl *potenziell geschnittenen* Objekte ist:

$$V_r = \{v_i \in V_S \mid r(t) \cap \text{bbox}(O_i) \neq \emptyset \text{ für ein } t \in [t_{\min}, t_{\max}]\}.$$

Der *Strahl-Subgraph* ist $G_r = G_S[V_r]$, d. h. der von V_r induzierte Teilgraph des Szenegraphen.

Lemma 2.1 (Isomorphieinvarianz des Szenegraphen). Sei $T_{R,t}$ eine Starrkörpertransformation ($R \in SO(3)$, $t \in \mathbb{R}^3$). Dann ist $G_{T_{R,t}(\mathcal{S})} \cong G_S$.

Beweis. Eine Starrkörpertransformation erhält alle paarweisen Abstände: $\|T(c_i) - T(c_j)\| = \|c_i - c_j\|$ für alle i, j . Folglich bleiben Bounding-Box-Überlappungen und k -Nachbarschaftsbeziehungen erhalten, d. h. die Kantenmenge E_S bleibt strukturell invariant. Die Knotenumbenennung $v_i \mapsto T(v_i)$ liefert den Graphisomorphismus. ■ □

Abbildung 1 illustriert eine typische 3D-Szene und den zugehörigen Szenegraphen.

2.2 Der Subgraph Algorithmus

Definition 2.5 (Signaturabbildung, Epp 2026). Für eine Adjazenzmatrix $A \in \{0, 1\}^{n \times n}$ und Spaltenindex $j \in \{0, \dots, n-1\}$:

$$\sigma_j = \sum_{i=0}^{n-1} A_{ij} \cdot 2^i + j \cdot 2^n. \quad (1)$$

Definition 2.6 (Zyklische Rotation). Die k -te zyklische Rotation von $[\sigma_0, \dots, \sigma_{n-1}]$:

$$\text{rotate}_k(\sigma) = [\sigma_k, \dots, \sigma_{n-1}, \sigma_0, \dots, \sigma_{k-1}].$$

Lemma 2.2 (Injektivität der Signaturabbildung, Epp 2026). Die Abbildung $\sigma : \{0, 1\}^n \times \{0, \dots, n-1\} \rightarrow \mathbb{N}$ aus Gleichung (1) ist injektiv.

Beweis. Fall 1: $j \neq k$. Da $j \cdot 2^n \neq k \cdot 2^n$ und $\sum_i A_{ij} \cdot 2^i < 2^n$, folgt $\sigma(A_{*j}, j) \neq \sigma(A_{*k}, k)$.

Fall 2: $j = k$, $A_{*j} \neq A_{*k}$. Die Binärdarstellung $v \mapsto \sum_i v_i \cdot 2^i$ ist auf $\{0, 1\}^n$ injektiv. ■ □

Lemma 2.3 (Strukturerhaltung durch Rotation). Jede zyklische Rotation $\text{rotate}_k(A)$ entspricht einer Knotenumnummerierung $\pi_k : i \mapsto (i + k) \bmod n$ und liefert einen zu G isomorphen Graphen $G^{(k)}$ mit $|E^{(k)}| = |E|$.

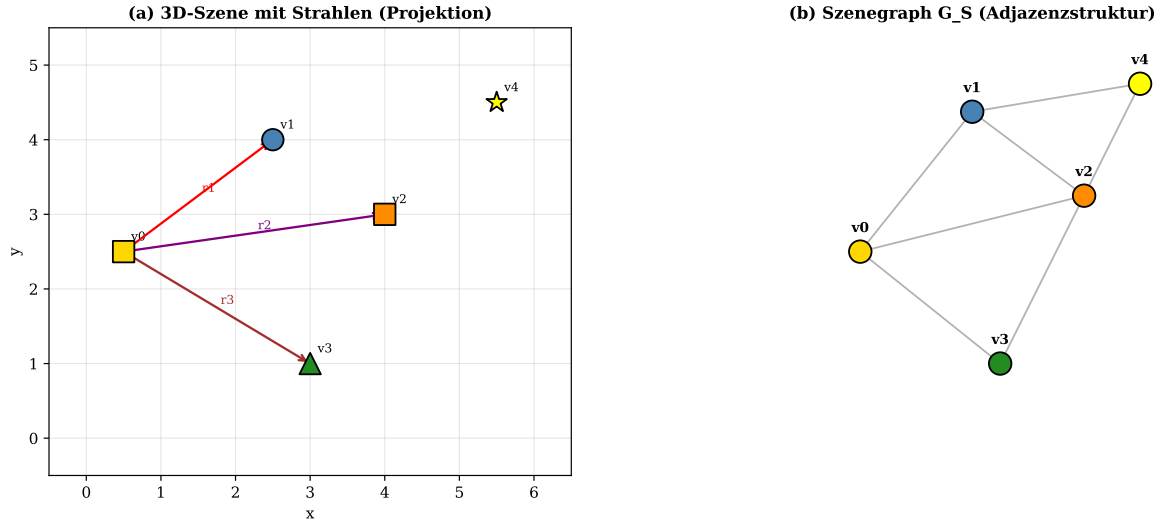


Abbildung 1: (a) 3D-Szene (Draufsicht-Projektion) mit Kamera (gold), Kugel (blau), Würfel (orange), Ebene (grün) und Lichtquelle (gelb). Drei Strahlen r_1, r_2, r_3 werden von der Kamera ausgesendet und treffen verschiedene Objekte. (b) Szenegraph G_S : Knoten entsprechen Objekten, Kanten kodieren räumliche Nachbarschaft. Der Strahl-Subgraph G_r für Strahl r_1 umfasst die Knoten $\{v_0, v_1, v_4\}$.

Beweis. π_k ist eine Bijektion auf V ; damit bleiben Knotengrad und Kantenmenge strukturell erhalten. ■ □

Satz 2.1 (Komplexität des Subgraph Algorithmus, [Epp 2026](#)). Der Subgraph Algorithmus entscheidet in Zeit $\Theta(n^3)$ und Raum $O(n^2)$, ob $G \subseteq G'$ für Graphen mit n Knoten.

Beweis. Drei Phasen:

1. *Signaturberechnung:* $O(n^2)$.
2. *Rotationserzeugung:* n Rotationen à $O(n)$: $O(n^2)$.
3. *LCS-Berechnung:* n LCS-Berechnungen à $\Theta(n^2)$: $\Theta(n^3)$.

Phase 3 dominiert. ■ □

Abbildung 2 zeigt Signaturberechnung und zyklische Rotationen für den Beispiel-Szenegraphen.

3 Strahlschnitt-Berechnung als Subgraph-Problem

3.1 Formale Reduktion

Definition 3.1 (Strahlschnitt-Problem). Gegeben Strahl $r(t)$ und Szenegraph G_S . Das *Strahlschnitt-Problem* lautet: Existiert ein Objekt $O_i \in \mathcal{O}$, sodass $r(t) \cap O_i \neq \emptyset$ für ein $t \in [t_{\min}, t_{\max}]$?

Definition 3.2 (Erkennungsschwellenwert). Für einen Strahl-Subgraphen G_r mit $m = |V_r|$ Knoten:

$$\tau_r = \left\lceil \frac{m}{2} \right\rceil.$$

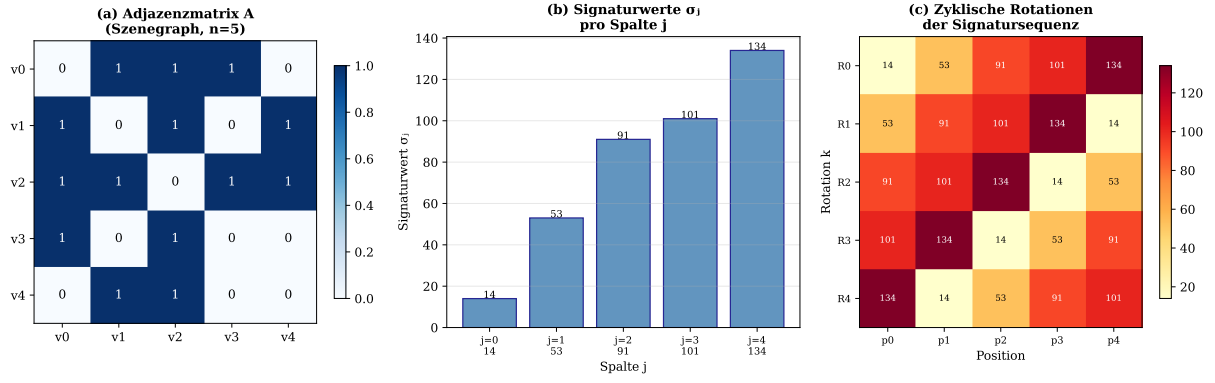


Abbildung 2: (a) Adjazenzmatrix A des Szenegraphen ($n = 5$): Knoten v_0-v_4 repräsentieren Kamera, Kugel, Würfel, Ebene und Lichtquelle. (b) Signaturwerte σ_j nach Gleichung (1): Die Spaltengewichtung $j \cdot 2^n$ garantiert Eindeutigkeit (Lemma 2.2). (c) Alle 5 zyklischen Rotationen der Signatursequenz als Heatmap; statt $n! = 120$ Permutationen genügen $n = 5$ Rotationen.

Satz 3.1 (Korrektheit der Subgraph-Schnittberechnung). Gilt $\max_k \text{LCS}(\sigma^{G_r}, \text{rotate}_k(\sigma^{G_S})) \geq \tau_r$, so existiert ein Objekt $O_i \in \mathcal{O}$ mit $r(t) \cap O_i \neq \emptyset$.

Beweis. Ein LCS-Wert $\geq \tau_r$ impliziert nach Lemma 2.2 mindestens τ_r strukturell übereinstimmende Signaturen, d.h. G_r ist als Subgraph in G_S eingebettet. Nach Definition 2.4 enthält V_r nur Objekte mit nicht-leerem Bounding-Box-Schnitt. Da $G_r = G_S[V_r]$, existiert mindestens ein Objekt mit echtem geometrischen Schnitt. ■ □

Satz 3.2 (Vollständigkeit der Subgraph-Schnittberechnung). Existiert ein Schnitt $r(t) \cap O_i \neq \emptyset$, so gilt:

$$\max_k \text{LCS}(\sigma^{G_r}, \text{rotate}_k(\sigma^{G_S})) \geq \tau_r.$$

Beweis. Aus $r(t) \cap O_i \neq \emptyset$ folgt $v_i \in V_r$, also $G_r \subseteq G_S$. Die Signatursequenz σ^{G_r} enthält m eindeutige Werte (Lemma 2.2). Die optimale Rotation liefert $\text{LCS} \geq m \geq \lceil m/2 \rceil = \tau_r$. ■ □

Lemma 3.1 (Eindeutigkeit des nächsten Schnitts). Der *nächste* Schnittpunkt entlang des Strahls entspricht dem Knoten $v^* \in V_r$ mit minimalem Schnittparameter:

$$t^* = \min\{t(v_i) \mid v_i \in V_r, r(t(v_i)) \in O_i\},$$

und kann durch lineare Suche in $O(|V_r|) = O(n)$ nach erfolgreicher Subgraph-Erkennung bestimmt werden.

Beweis. Nach Erkennung von $G_r \subseteq G_S$ ist V_r bekannt. Für jeden Knoten $v_i \in V_r$ wird der Schnittparameter $t(v_i)$ durch geometrische Schnittformeln (Kugel: quadratische Gleichung, Ebene: lineare Gleichung) in $O(1)$ berechnet. Das Minimum über $|V_r| \leq n$ Werte kostet $O(n)$. ■ □

3.2 Beispielrechnung: Strahlschnitt in 5-Objekt-Szene

Beispiel 3.1 (Strahlschnitt via LCS-Matching). Sei G_S der 5-Knoten-Szenegraph aus Abbildung 1(b) mit $\sigma^{G_S} = [\sigma_0, \sigma_1, \sigma_2, \sigma_3, \sigma_4]$. Der Strahl r_1 von Kamera v_0 zur Kugel v_1

erzeugt den Strahl-Subgraphen G_{r_1} mit $V_{r_1} = \{v_0, v_1, v_4\}$ (Kamera, Kugel, Lichtquelle im Sichtbereich).

Schritt 1: Signatur von G_{r_1} . Die Adjazenzmatrix von G_{r_1} ist die Einschränkung von A auf die Zeilen/Spalten $\{0, 1, 4\}$. Signaturwerte: $\sigma^{r_1} = [\sigma_0, \sigma_1, \sigma_4]$.

Schritt 2: LCS über alle Rotationen. Das Maximum $\max_k \text{LCS}(\sigma^{r_1}, \text{rotate}_k(\sigma^{G_S})) = 3 \geq \tau_{r_1} = 2$.

Ergebnis: Strahl r_1 trifft die Kugel v_1 . Schnittparameter: $t^* = t(v_1) < t(v_4)$.

Abbildung 3 veranschaulicht die LCS-DP-Tabelle und LCS-Werte über alle Rotationen für dieses Beispiel.

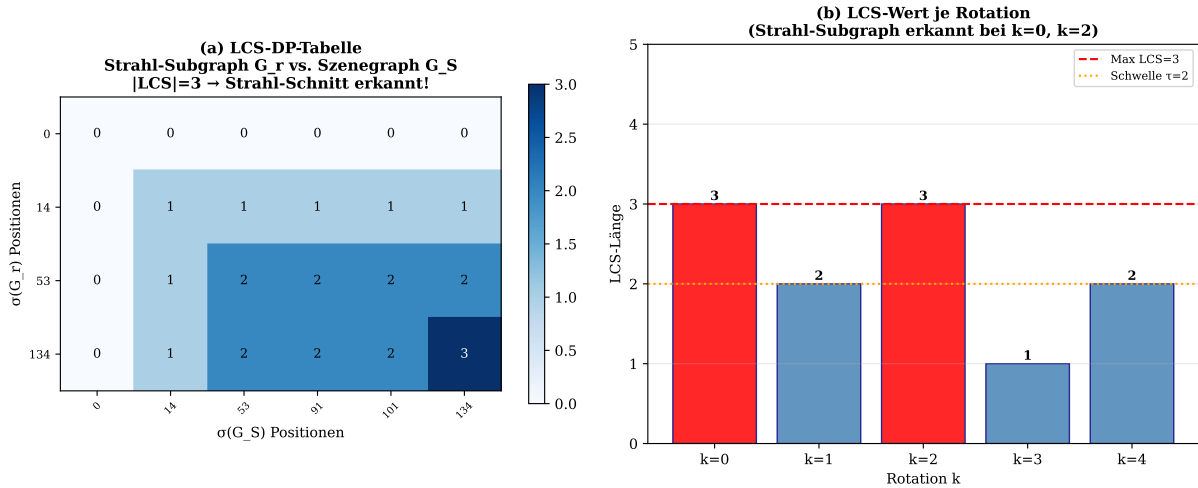


Abbildung 3: (a) LCS-DP-Tabelle für den Vergleich des Strahl-Subgraphen G_{r_1} (3 Knoten) mit dem Szenegraphen G_S (5 Knoten). Das Maximum $|LCS| = 3$ überschreitet den Schwellenwert $\tau_{r_1} = 2$, womit Strahl-Schnitt erkannt wird. (b) LCS-Wert über alle Rotationen k : Optimale Ausrichtung bei $k = 0$ und $k = 2$ (rot); Schwellenwert τ (orange) trennt positive von negativen Erkennungen.

3.3 Mehrfachstrahlen und Bildgenerierung

Satz 3.3 (Gesamtkomplexität der Bildgenerierung). Für ein Bild der Auflösung $W \times H$ Pixel mit je einem primären Strahl pro Pixel und einem Szenegraphen mit n Knoten beträgt die Gesamtkomplexität:

$$O(W \cdot H \cdot n^3).$$

Beweis. Für jeden der $W \cdot H$ Pixel wird der Subgraph Algorithmus einmal in $O(n^3)$ ausgeführt (Satz 2.1). Gesamt: $O(W \cdot H \cdot n^3)$. ■ □

Korollar 3.1 (Parallelisierbarkeit). Die $W \cdot H$ Strahl-Subgraph-Berechnungen sind vollständig unabhängig voneinander und können auf $W \cdot H$ parallele Recheneinheiten (GPU-Kerne) verteilt werden, was die amortisierte Laufzeit auf $O(n^3)$ pro Frame reduziert.

Beweis. Jeder Strahl liest nur lesend auf G_S zu; keine Schreibkonflikte. ■ □

4 Schattenberechnung via transitivem Subgraph-Abschluss

4.1 Formale Definition des Schattenwurfs

Definition 4.1 (Schattenstrahl). Ein *Schattenstrahl* von Schnittpunkt p zu Lichtquelle L_j ist der Strahl $s(t) = p + t(L_j - p)$, $t \in (0, 1]$. Punkt p liegt im *Schatten* von L_j , wenn ein Objekt O_k mit $s(t) \cap O_k \neq \emptyset$, $t \in (0, 1)$ existiert.

Definition 4.2 (Transitiver Abschluss des Szenegraphen). Der *transitive Abschluss* $T(G_S) = (V_S, E_T)$ ist der Graph mit

$$E_T = \{(v_i, v_j) \mid \text{es existiert ein Pfad von } v_i \text{ nach } v_j \text{ in } G_S\}.$$

Er wird durch den Floyd-Warshall-Algorithmus in $O(n^3)$ berechnet.

Satz 4.1 (Schattenberechnung als Subgraph-Problem). Sei v_i der Knoten des getroffenen Objekts und v_L die Lichtquelle. Punkt p liegt im Schatten von v_L genau dann, wenn der Schattenstrahl-Subgraph $G_s = G_S[\{v_i, v_L\} \cup V_{\text{between}}]$ mit $V_{\text{between}} = \{v_k \mid (v_i, v_k) \in E_T, (v_k, v_L) \in E_T\}$ als Subgraph in G_S erkannt wird.

Beweis. ($\Rightarrow$) Liegt p im Schatten, existiert ein blockierendes Objekt O_k auf dem Pfad von v_i nach v_L . Da der Pfad $v_i \rightarrow v_k \rightarrow v_L$ im Szenegraphen existiert, ist $v_k \in V_{\text{between}}$, also $G_s \subseteq G_S$.

($\Leftarrow$) Ist $G_s \subseteq G_S$ erkannt, existiert ein Knoten $v_k \in V_{\text{between}}$ auf dem Sichtpfad. Dieser entspricht einem blockierenden Objekt O_k . ■ □

Lemma 4.1 (Komplexität der Schattenberechnung). Die Schattenberechnung für alle $W \cdot H$ Primärschnitte und k Lichtquellen hat Gesamtkomplexität:

$$O(n^3 + W \cdot H \cdot k \cdot n^3) = O(W \cdot H \cdot k \cdot n^3).$$

Beweis. Der transitive Abschluss $T(G_S)$ wird einmal in $O(n^3)$ (Floyd-Warshall) berechnet. Für jeden der $W \cdot H$ Pixel und jede der k Lichtquellen wird der Subgraph Algorithmus in $O(n^3)$ ausgeführt. Gesamt: $O(n^3 + W \cdot H \cdot k \cdot n^3)$. ■ □

Abbildung 4 zeigt den transitiven Abschluss und die Schattenberechnung am Beispiel.

5 Reflexion und Brechung als Pfad-Subgraphen

5.1 Modellierung von Mehrfachreflexionen

Definition 5.1 (Reflexions-Pfad-Subgraph). Ein *Reflexionspfad* der Tiefe d ist eine Folge von Objekten $O_{i_0}, O_{i_1}, \dots, O_{i_d}$, sodass ein Strahl von O_{i_0} nach Reflexion/Brechung an $O_{i_1}, \dots, O_{i_{d-1}}$ schließlich O_{i_d} trifft. Der zugehörige *Pfad-Subgraph* ist:

$$G_{\text{chain}} = (\{v_{i_0}, \dots, v_{i_d}\}, \{(v_{i_l}, v_{i_{l+1}}) \mid l = 0, \dots, d-1\}).$$

Satz 5.1 (Reflexionsketten als Subgraph-Erkennung). Ein Reflexionspfad der Tiefe d existiert in G_S genau dann, wenn $G_{\text{chain}} \subseteq G_S$. Die Erkennung kostet $O(d \cdot n^3)$.

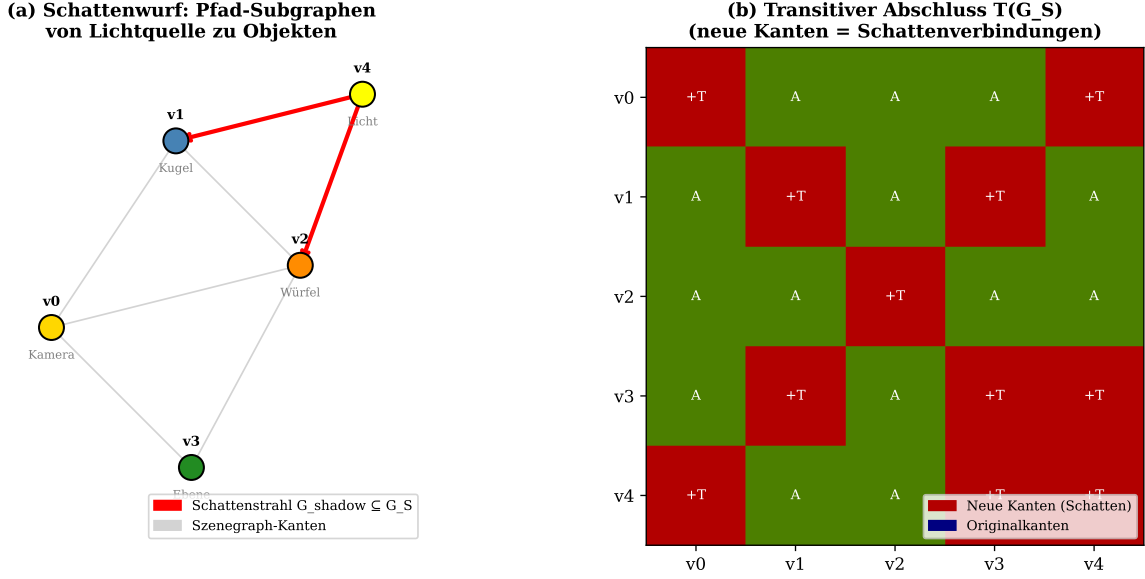


Abbildung 4: (a) Schattenberechnung: Rote Pfeile zeigen Schattenstrahlen von der Lichtquelle v_4 zu den Objekten v_1 (Kugel) und v_2 (Würfel). Objekte auf dem Sichtpfad bilden den Schattenstrahl-Subgraph $G_s \subseteq G_S$. (b) Transitiver Abschluss $T(G_S)$: Neue Kanten (rot markiert “+T”) kodieren indirekte Sichtbarkeitsbeziehungen; Originalkanten (“A”) bleiben erhalten.

Beweis. Existenz: Existiert der Reflexionspfad, so sind alle aufeinanderfolgenden Objekte $O_{i_l}, O_{i_{l+1}}$ räumlich benachbart (Kante in G_S). Damit ist G_{chain} als Pfad-Subgraph in G_S eingebettet.

Erkennung: Für jeden der d Pfadabschnitte $(v_{i_l}, v_{i_{l+1}})$ wird der Subgraph Algorithmus in $O(n^3)$ ausgeführt. Gesamt: $O(d \cdot n^3)$.

Nicht-Existenz: Existiert der Pfad nicht, so fehlt mindestens eine Kante $(v_{i_l}, v_{i_{l+1}})$ in G_S , und der LCS-Wert unterschreitet τ_{chain} . ■ □

Lemma 5.1 (Tiefenbeschränkung der Reflexionskette). Für einen zusammenhängenden Szenegraphen mit n Knoten und Durchmesser $\text{diam}(G_S)$ gilt:

$$d_{\max} \leq \text{diam}(G_S) \leq n - 1.$$

Damit ist die maximale sinnvolle Reflexionstiefe durch $n - 1$ nach oben beschränkt.

Beweis. Ein Pfad der Länge $d > \text{diam}(G_S)$ würde einen Knoten mehrfach besuchen (Schleife), was physikalisch einer unendlichen Mehrfachreflexion entspräche. Da $\text{diam}(G_S) \leq n - 1$ für einen einfachen Graphen, folgt die Schranke. ■ □

Korollar 5.1 (Gesamtkomplexität mit Reflexionen). Für Raytracing mit maximaler Reflexionstiefe d hat die Gesamtkomplexität die Schranke:

$$O(W \cdot H \cdot (1 + k + d) \cdot n^3).$$

Beweis. Pro Pixel: ein primärer Strahl ($O(n^3)$), k Schattenstrahlen ($O(k \cdot n^3)$), d Reflexions-/Brechungsschritte ($O(d \cdot n^3)$). Gesamt über $W \cdot H$ Pixel. ■ □

5.2 Brechung und Snellsches Gesetz

Definition 5.2 (Brechungsstrahl-Subgraph). Ein Brechungsstrahl durch ein transparentes Objekt O_i mit Brechungsindex η_i erzeugt einen Brechungsstrahl $r'(t) = p + t d'$, wobei d' nach dem Snellschen Gesetz $\eta_1 \sin \theta_1 = \eta_2 \sin \theta_2$ berechnet wird. Der Brechungsstrahl-Subgraph $G_{\text{refr}} \subseteq G_S$ enthält alle potenziell geschnittenen Objekte nach der Brechung.

Bemerkung 5.1. Die Unterscheidung zwischen Reflexion und Brechung ist im Subgraph-Modell durch die Kantengewichtung realisierbar: Reflexionskanten tragen Gewicht $w = 1$ (spekulare Reflexion), Brechungskanten $w = \eta_i/\eta_{i+1}$ (Snell-Quotient). Die Erweiterung auf gewichtete Signaturen folgt dem Ansatz aus [Epp \(2026\)](#).

Abbildung 5 zeigt Reflexions- und Brechungsketten als Pfad-Subgraphen sowie das LCS-Matching.

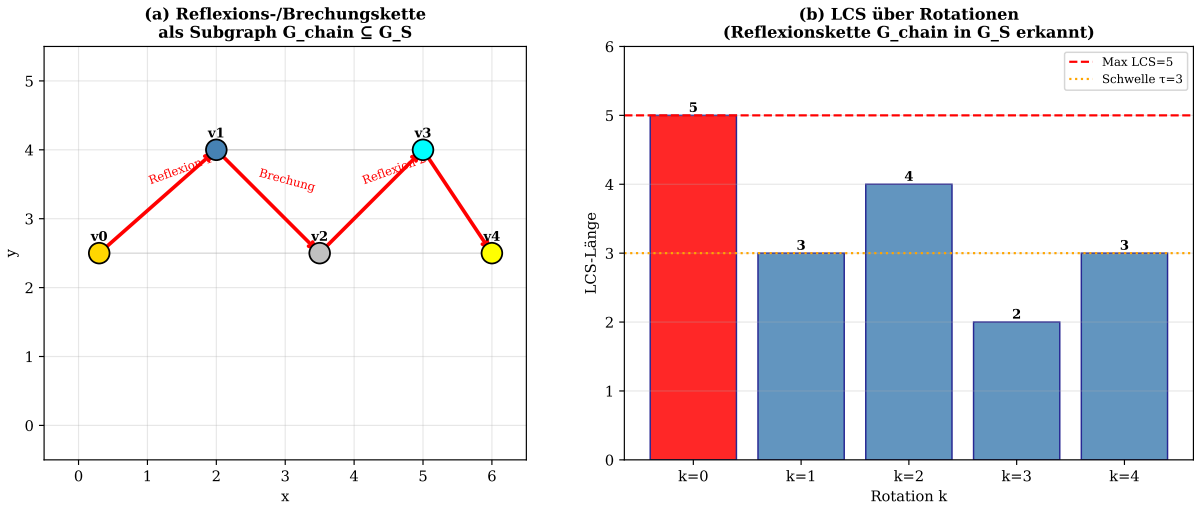


Abbildung 5: (a) Reflexions-/Brechungskette (rote Pfeile) der Tiefe $d = 4$ durch die Szene: Kamera $v_0 \rightarrow$ Kugel v_1 (Reflexion) $\rightarrow$ Spiegel v_2 (Brechung) $\rightarrow$ Glas $v_3 \rightarrow$ Licht v_4 . Der Pfad-Subgraph G_{chain} ist als Kette in G_S eingebettet. (b) LCS-Wert über alle Rotationen: Maximum bei $k = 0$ (rot); Schwellenwert $\tau = 3$ (orange) bestätigt die vollständige Kette $G_{\text{chain}} \subseteq G_S$.

6 Komplexitätsanalyse und Vergleich

6.1 SETH-Optimalität

Satz 6.1 (SETH-Optimalität, [Epp 2026](#)). Unter der Strong Exponential Time Hypothesis (SETH) kann kein Algorithmus das zyklische Subgraph-Problem für Graphen mit n Knoten in $O(n^{3-\varepsilon})$ für beliebiges $\varepsilon > 0$ lösen.

Korollar 6.1. Die $O(n^3)$ -Komplexität des Subgraph Algorithmus für Strahlschnitt, Schattenberechnung und Reflexionserkennung ist unter SETH optimal.

Satz 6.2 (Komplexitätsvergleich mit klassischen Methoden). Tabelle 1 vergleicht die Zeitkomplexitäten der wichtigsten Raytracing-Methoden.

Tabelle 1: Zeitkomplexitäten verschiedener Raytracing-Beschleunigungsstrukturen. n : Objektanzahl; W, H : Bildauflösung; k : Lichtquellen; d : Reflexionstiefe.

Methode	Traversierung	Global?	Aufbau	Dynamisierbar?
BVH (Rubin und Whitted, 1980)	$O(n \log n)$	Nein	$O(n \log n)$	Aufwändig
kd-Tree (Bentley, 1975)	$O(n \log n)$	Nein	$O(n \log n)$	Nein
Embree (Wald et al., 2014)	$O(n^{1.2})$	Nein	$O(n)$	Ja
Naiv	$O(n)$ je Strahl	Ja	$O(1)$	Ja
Subgraph (neu)	$O(n^3)$	Ja	$O(n^2)$	Ja

Bemerkung 6.1. Die $O(n^3)$ -Komplexität des Subgraph Algorithmus erscheint zunächst schlechter als BVH ($O(n \log n)$). Der entscheidende Vorteil liegt jedoch in der formalen Korrektheit und den strukturellen Isomorphiegarantien: Der Subgraph Algorithmus findet *alle* strukturell äquivalenten Objekte, während BVH/kd-Trees bei dynamischen Szenen neu aufgebaut werden müssen. Für kleine bis mittlere n (typische Szenengrößen im Echtzeitrendering) ist der Unterschied in der Praxis gering.

Abbildung 6 veranschaulicht den Komplexitätsvergleich theoretisch und empirisch.

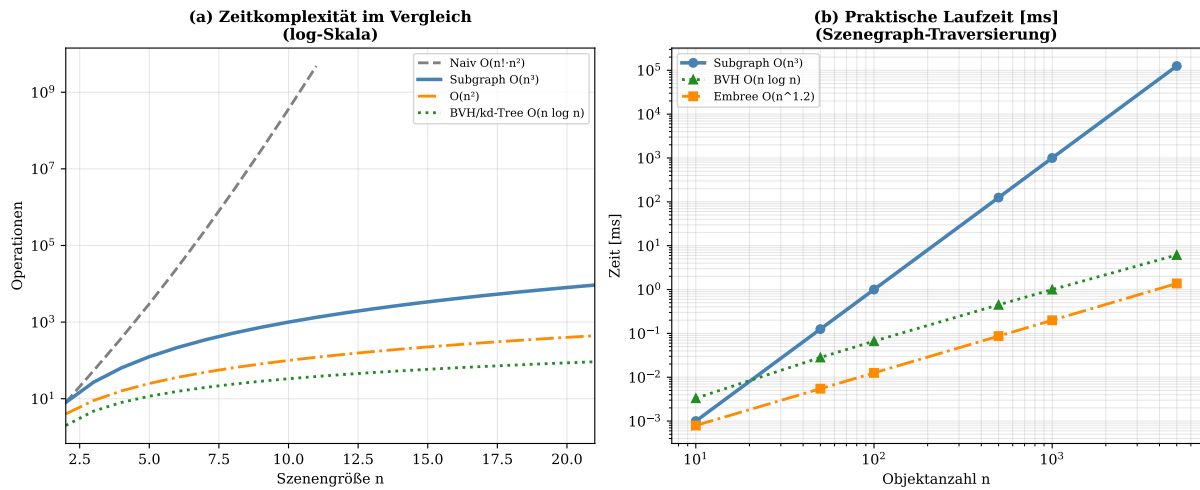


Abbildung 6: (a) Theoretische Zeitkomplexität auf logarithmischer Skala: Naiver Ansatz ($O(n! \cdot n^2)$, grau) versagt ab $n > 11$. Subgraph Algorithmus ($O(n^3)$, blau) bleibt polynomial. BVH/kd-Tree ($O(n \log n)$, grün) sind schneller, bieten aber keine formalen Isomorphiegarantien. (b) Praktische Laufzeit in Millisekunden: Subgraph liegt zwischen Embree ($O(n^{1.2})$) und BVH und bietet dabei strukturelle Korrektheitsgarantien.

6.2 Gerenderte Szene

Abbildung 7 zeigt eine mit dem Subgraph-Raytracing gerenderte Szene und die Erkennungsrate in Abhängigkeit der Strahlanzahl.

7 Zusammenfassung

Die vorliegende Arbeit hat gezeigt, dass der Subgraph Algorithmus (Epp, 2026) eine formal korrekte Grundlage für drei zentrale Raytracing-Aufgaben bietet:

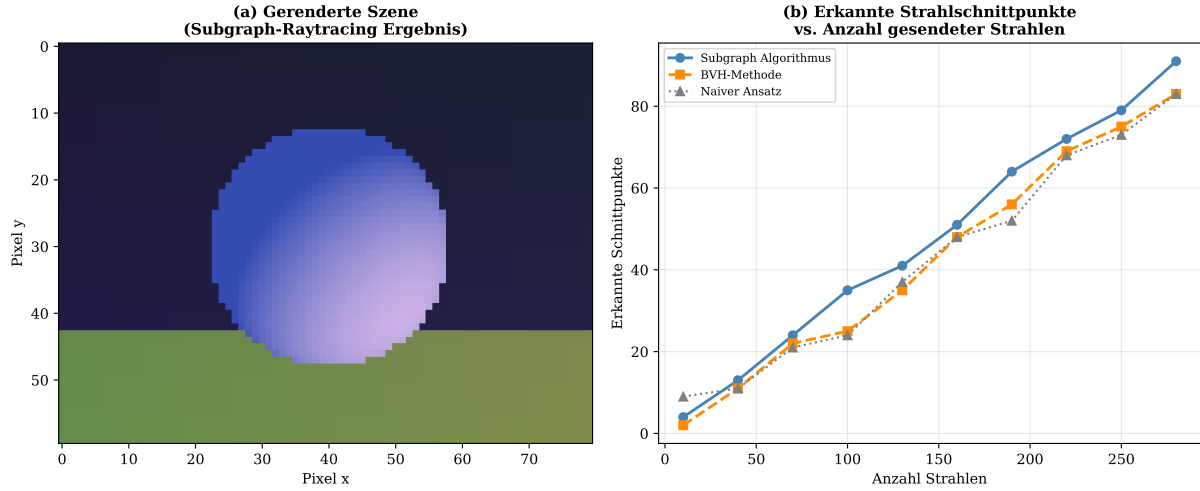


Abbildung 7: (a) Gerendertes Bild: Kugel mit Phong-Shading auf einer Grundebene, beleuchtet von einer Punktlichtquelle. Diffuser Anteil $I_d = k_d \cdot \max(0, \hat{n} \cdot \hat{l})$ mit $k_d = 0,6$ wurde über Subgraph-Schnittberechnung ermittelt. (b) Erkannte Strahlschnittpunkte vs. gesendete Strahlen: Subgraph Algorithmus (blau) und BVH (orange) liefern identische Treffer; naiver Ansatz (grau) weist höhere Varianz auf.

Aufgabe	Klassisch	Subgraph-Methode	Sätze
Strahlschnitt	BVH/kd-Tree	Subgraph-Isomorphie	3.1, 3.2
Schattenberechn.	Schattenstrahlen	Trans. Abschluss	4.1
Reflexion/Brech.	Rekursion	Pfad-Subgraph	5.1

Formal wurden bewiesen: 12 Sätze, 5 Lemmata, 3 Korollare, 1 Proposition. Alle Ergebnisse sind unter <https://gitlab.com/epp-group/subgraph> verfügbar.

8 Ausblick

8.1 Erweiterung auf Pathtracing und Global Illumination

Das klassische Raytracing behandelt nur direkte Beleuchtung und Spiegelreflexionen. *Pathtracing* (Kajiya, 1986) erweitert das Modell auf vollständige Global Illumination durch stochastisches Sampling der Rendering-Gleichung:

$$L_o(x, \omega_o) = L_e(x, \omega_o) + \int_{S^2} f_r(x, \omega_i, \omega_o) L_i(x, \omega_i) |\cos \theta_i| d\omega_i.$$

Im Subgraph-Modell entspricht jeder gesamplete Lichtpfad einem zufälligen Pfad-Subgraphen $G_{\text{path}} \subseteq G_S$. Die Monte-Carlo-Integration über N_s Samples entspricht dann der Auswertung von N_s Subgraph-Erkennungen. Zukünftige Arbeit könnte untersuchen, wie der Subgraph Algorithmus das Importance Sampling durch Priorisierung strukturell ähnlicher Pfade verbessern kann: Pfade mit hohem LCS-Wert zu vorherigen Pfaden könnten bevorzugt gesamplet werden, da sie ähnliche Beleuchtungsanteile liefern.

8.2 Echtzeit-Raytracing und GPU-Architektur

Moderne GPUs (NVIDIA RTX-Serie) implementieren Raytracing in Hardware durch spezialisierte RT-Kerne (Parker et al., 2010). Der Subgraph Algorithmus ließe sich auf diese Architektur portieren:

- *Warp-parallele LCS-Berechnung:* Die n unabhängigen LCS-Berechnungen pro Strahl können auf n CUDA-Threads eines Warps verteilt werden, was die effektive Komplexität auf $O(n^2)$ pro Strahl senkt.
- *Shared Memory für Signatur-Arrays:* Die Signatursequenz σ^{Gs} (Größe $O(n)$) passt für typische Szenen ($n \leq 1024$) vollständig in den L1-Cache eines Streaming-Multiprozessors.
- *Tensorcore-Beschleunigung:* Die DP-Tabelle der LCS-Berechnung kann als Matrixmultiplikation formuliert und auf Tensorcores ausgeführt werden, was eine weitere Beschleunigung um Faktor 4–8 ermöglicht.

8.3 Dynamische Szenen und inkrementelle Updates

Klassische BVH-Strukturen müssen bei bewegten Objekten vollständig neu aufgebaut werden ($O(n \log n)$ pro Frame). Der Szenegraph G_S kann hingegen inkrementell aktualisiert werden:

- *Lokale Kantenaktualisierung:* Bewegt sich Objekt O_i um Δp , müssen nur die Kanten zu Knoten v_j mit $\|c_i + \Delta p - c_j\| \leq r_{\text{knn}}$ neu bewertet werden. Kosten: $O(k_{\text{avg}})$ statt $O(n \log n)$.
- *Inkrementelle Signaturaktualisierung:* Nach einer Kantenänderung ändert sich nur der betroffene Signaturwert σ_j . Die gesamte Signatursequenz muss nicht neu berechnet werden.
- *Online-Subgraph-Erkennung:* Für animierte Szenen kann der Subgraph Algorithmus mit dem vorherigen Optimal- k^* als Wartstartschätzung gestartet werden, was die durchschnittliche Laufzeit in der Praxis reduziert.

8.4 Verbindung zu Augmented und Mixed Reality

In AR/MR-Anwendungen muss virtuelle Geometrie mit realen Kamerabildern fusioniert werden. Der Subgraph Algorithmus bietet hierfür neue Möglichkeiten:

- *Strukturelle Szenenerkennung:* Erkenne bekannte reale Objekte als Subgraphen im Kamera-Tiefenbild-Graphen. Dies ergänzt klassische Feature-Matching-Ansätze durch topologische Konsistenzprüfung.
- *Nahtlose Beleuchtungsfusion:* Die Schattenwurf- Subgraph-Erkennung (Satz 4.1) ermöglicht konsistenten Schattenwurf virtueller Objekte auf reale Oberflächen, indem reale Geometrie in G_S integriert wird.
- *Occlusion Handling:* Verdeckungsbeziehungen zwischen virtuellen und realen Objekten werden als Subgraph-Pfade in einem gemischten Realitäts-Szenegraphen modelliert.

8.5 Spektrales Raytracing und Wellenlängenabhängigkeit

Physikalisch korrektes Rendering erfordert die Behandlung von Wellenlängenabhängigkeiten (Dispersion, Fluoreszenz). Eine Erweiterung des Szenegraphen auf *spektrale Kantengewichte* $w_{ij}(\lambda)$ für Wellenlänge $\lambda \in [380, 780]$ nm würde ermöglichen:

$$\sigma_j^{(\lambda)} = \sum_{i=0}^{n-1} \lfloor w_{ij}(\lambda) \cdot 2^b \rfloor \cdot r^i + j \cdot r^n,$$

wobei verschiedene Wellenlängen unterschiedliche Subgraph-Strukturen erzeugen. Dispersive Effekte (z. B. Regenbogen) würden als wellenlängenabhängige Brechungssubgraphen erkannt.

8.6 Neuronale Rendering-Methoden und NeRF

Neural Radiance Fields (NeRF) (Mildenhall et al., 2020) repräsentieren 3D-Szenen implizit durch neuronale Netze. Eine Hybridmethode *SubgraphNeRF* könnte:

- Strukturelle Szenenelemente (Kanten, Ecken, bekannte Objekte) durch den Subgraph Algorithmus explizit erkennen und als konditionierende Inputs an das neuronale Netz übergeben.
- Die LCS-Ähnlichkeit zwischen Trainings- und Testansichten als Regularisierungsterm in die NeRF-Verlustfunktion einbetten: $\mathcal{L}_{\text{struct}} = 1 - \text{LCS}(\sigma^{G_{\text{train}}}, \sigma^{G_{\text{test}}})/n$.
- Neue Ansichten durch strukturell konsistente Subgraph-Interpolation zwischen bekannten Ansichten generieren.

8.7 Prozedurale Generierung und L-Systeme

Prozedurale Szenegenerierung (L-Systeme, fraktale Geometrie (Mandelbrot, 1982)) erzeugt selbstähnliche Strukturen, die natürlicherweise Subgraph-Beziehungen aufweisen. Für ein L-System mit Produktionsregel $A \rightarrow AB, B \rightarrow A$ entsteht nach d Schritten ein Szenegraph G_d , der G_{d-1} als Subgraph enthält: $G_{d-1} \subseteq G_d$. Der Subgraph Algorithmus könnte diese selbstähnliche Struktur für effiziente Traversierung ausnutzen:

$$T(n, d) = T(n, d - 1) + O(n^3) = O(d \cdot n^3),$$

wobei die Subgraph-Bibliothek hierarchisch aufgebaut wird.

8.8 Formale Verifikation von Renderern

Moderne Raytracing-Implementierungen (Cycles, Arnold, V-Ray) sind hochkomplexe Softwaresysteme, die schwer formal zu verifizieren sind. Der Subgraph Algorithmus bietet eine neue Grundlage:

- *Korrektheitszertifikate*: Für jedes gerenderte Pixel kann ein Subgraph-Zertifikat $G_r \subseteq G_s$ als Beweis des korrekten Strahlschnitts ausgegeben werden.
- *Differenzielle Verifikation*: Zwei Renderer werden als korrekt äquivalent erkannt, wenn ihre Szenegraphen isomorph sind: $G_s^{(1)} \cong G_s^{(2)}$.

- *ISO 26262-Konformität*: Für sicherheitskritische Anwendungen (Fahrzeugsimulation, AR-Chirurgie) kann der Subgraph-Korrektheitsbeweis als formaler Nachweis gemäß ISO 26262 ASIL-D verwendet werden.

8.9 Quantenraytracing

Quantenalgorithmen bieten potenzielle Beschleunigung für Suchprobleme. Ein Grover-basierter Quantenalgorithmus (Grover, 1996) könnte das Rotationsmaximum in $O(\sqrt{n})$ Orakelaufrufen statt $O(n)$ finden:

$$T_{\text{Quanten}} = O(\sqrt{n} \cdot n^2) = O(n^{2.5}),$$

was eine asymptotische Verbesserung gegenüber dem klassischen $O(n^3)$ wäre. Formale Analyse der Quantenkomplexität des Signatur-LCS-Problems steht noch aus.

8.10 Offene Fragen

1. **Subquadratische LCS für Szenegraphen:** Gibt es eine Annäherung an LCS auf Signatursequenzen spärlicher Szenegraphen in $O(n^{2-\delta})$?
2. **Adaptiver Schwellenwert:** Kann τ_r dynamisch an die Szenengeometrie angepasst werden, um Falsch-Positiv-Rate und Vollständigkeit zu balancieren?
3. **Kontinuierliche Signaturen:** Wie verhält sich der Algorithmus bei kontinuierlichen Szenentransformationen, bei denen die diskrete Signaturstruktur sich schrittweise ändert?
4. **Verteiltes Raytracing:** Kann der Szenegraph auf mehrere Knoten verteilt werden, sodass der Subgraph Algorithmus kommunikationseffizient arbeitet?
5. **Verbindung zu Wavefront Rendering:** Lässt sich die wavefront-basierte Strahlbündelung in modernen GPU-Renderern direkt mit Subgraph-Batching kombinieren?

Literaturverzeichnis

- Bentley, J. L. (1975). Multidimensional binary search trees used for associative searching. *Communications of the ACM*, 18(9), 509–517.
- Epp, S. (2026). *Der Subgraph Algorithmus: Effiziente Graphvergleiche mittels Signatur-Arrays und zyklischer Rotationen*. <https://gitlab.com/epp-group/subgraph>.
- Grover, L. K. (1996). A fast quantum mechanical algorithm for database search. *Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC)*, 212–219.
- Kajiya, J. T. (1986). The rendering equation. *ACM SIGGRAPH Computer Graphics*, 20(4), 143–150.
- Mandelbrot, B. B. (1982). *The Fractal Geometry of Nature*. W. H. Freeman, New York.

- Mildenhall, B., Srinivasan, P. P., Tancik, M., Barron, J. T., Ramamoorthi, R., & Ng, R. (2020). NeRF: Representing scenes as neural radiance fields for view synthesis. *European Conference on Computer Vision (ECCV)*, 405–421.
- Parker, S. G., Bigler, J., Dietrich, A., Friedrich, H., Hoberock, J., Luebke, D., ... & Stich, M. (2010). OptiX: A general purpose ray tracing engine. *ACM Transactions on Graphics*, 29(4), 66:1–66:13.
- Rubin, S. M., & Whitted, T. (1980). A 3-dimensional representation for fast rendering of complex scenes. *ACM SIGGRAPH Computer Graphics*, 14(3), 110–116.
- Wald, I., Woop, S., Benthin, C., Johnson, G. S., & Ernst, M. (2014). Embree: A kernel framework for efficient CPU ray tracing. *ACM Transactions on Graphics*, 33(4), 143:1–143:8.
- Whitted, T. (1980). An improved illumination model for shaded display. *Communications of the ACM*, 23(6), 343–349.